

Proyecto 01 - Espora

Buendia Elizalde Ricardo	322010215
Alvarez Reyes Janeth Alin	322279832
Oropeza Zambrano Fernando	425013902

Para nuestro proyecto hicimos uso de **Java 17**, por lo que recomendamos tener esta versión instalada. En nuestro caso, trabajamos en **Linux**.

Argumentación

Para nuestro proyecto 1, se hizo uso de los siguientes patrones de diseño: Proxy, Strategy, State, Decorator y Observer. Ya que a nuestra consideración, fueron los más adecuados para poder implementar nuestras ideas.

Como en todo banco, la seguridad de nuestro dinero es lo mas importante, así que se nos pidió implementar un Acceso Remoto seguro para las cuentas, que, en su defecto, consiste en verificar un NIP valido como si fuera una transacción de un banco tradicional. Esto debe hacerse antes de permitir operaciones como depósitos, consultas de saldo, etc, y sin necesidad de alterar las funciones internas de la cuenta. Vimos que implementar el patrón Proxy era la mejor solución junto con RMI, de manera que la clase PortafolioServidor actuara como el intermediario, el cual intercepta las llamadas remotas, y ejecuta el método verificarNIP para validarlo con el que se tiene almacenado para el cliente dueño, y de ser exitosa la operación, se delega la operación, lo que ayuda a añadir una capa de seguridad que es muy necesaria en un banco.

Algo necesario que nos pedía que tuviera nuestro PumaBank, fue tener diferentes esquemas para calcular el interés (Mensual, Anual y Premium), los cuales deberán adaptarse a cada perfil de los clientes, vimos que una buena solución para esto era hacer uso del patrón Strategy, de forma que cada algoritmo para calcular el interés se encapsulara en su propia clase (PlanMensual, PlanAnual y PlanPremium) todas estas implementando una interfaz común que seria InteresEstrategia, así la clase CuentaCliente solo mantiene una referencia a InteresEstrategia y le delega la tarea de calcular el interés, sin tener que conocer los detalles específicos de cada plan.

Para la parte en que es necesario enviar notificaciones a los clientes, ya sea como el resumen mensual, o las alertas en tiempo real, vimos la solución mediante el patrón Observer, de forma que la CuentaCliente que actúa como Sujeto, mantiene una lista de suscriptores (Observador), y cuando ocurre un evento relevante como una transacción o el resumen mensual, la cuenta llama a su método notificarObservadores, lo que permite que la cuenta notifique a los clientes sin necesidad de conocerlos directamente.

Como en muchos bancos, los servicios adicionales van de la mano con estos, y en el proyecto se nos pidió ofrecer algunos como es el SeguroAntifraude, ProgramaRecompensas y AlertasPremium, los cuales deben de ser opcionales y combinables entre clientes, por eso vimos como una gran opción a Decorator, e hicimos la creacion de una clase abstracta ServicioDecorador, luego las clases concretas como lo son SeguroAntifraude, AlertasPremium y ProgramaRecompensas heredan de ServicioDecorador y añaden su comportamiento específico.

Para el patron State decidimos implementarlo en los distintos estados que puede tener una cuenta bancaria, ya sea Activa, Sobregirada, Congelada o Cerrada, debido a que cada uno define comportamientos distintos según las operaciones que el cliente puede realizar o no. De este modo los estados pueden modificarse dinámicamente según las acciones realizadas por el cliente en su cuenta.

Como necesitábamos definir la estructura de una cuenta bancaria, la cual debía poder manejar distintas operaciones, estados, etc. Creamos una interfaz cuenta para que pudiese actuar como el contrato principal, con los métodos que una cuenta debe de tener. Tenemos también una clase CuentaCliente que es la que mas métodos contiene pues es la implementación concreta de esa interfaz, aquí esta la lógica real, en la que se almacenan los atributos necesarios como saldo, EstadoActual, etc.

Creamos una clase LectorTXT que es el que se encarga de leer y procesar la información en nuestros 3 archivos .txt, de modo que así nuestro programa puede leer y utilizar esos datos para funcionar.

Esta entrega es tardía ya que no habíamos puesto suficientes casos en el main de clientes, por lo que no se apreciaba totalmente las cosas que puede hacer nuestro banco PumaBank, de igual manera sentimos que aun nos faltaron varios casos por mostrar, una disculpa.

Para compilar

Una vez que los archivos están descargados y ubicados en la carpeta Proyecto01.espora, ejecutar:

```
$ javac -d target src/main/java/mx/unam/fciencias/espora/proyecto1/*.java
```

Para ejecutar

Cuando se hayan generado los .class, ejecutar el main con dos terminales, primero:

```
$ java -cp target mx.unam.fciencias.espora.proyecto1.PumaBank
```

En una segunda terminal, haremos el siguiente comando:

```
$ java -cp target mx.unam.fciencias.espora.proyecto1.ClienteMain
```

Para compilar con Dockerfile

Estando en la misma carpeta Proyecto01_espora, primero ejecutar para construirlo:

```
$ docker build -t pumabank .
```

Y luego para correrlo ejecutar:

```
$ docker run --rm --network="host" --name puma pumabank
```

Estos dos primeros comandos van en la primer terminal.

Será necesario abrir otra terminal en su equipo para correr el siguiente comando:

```
$ docker run --rm -it --network="host" pumabank java -cp  
target mx.unam.fciencias.espora.proyecto1.ClienteMain
```

NOTA: Este ultimo comando va en la segunda terminal.